

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, con

```

```

data = load_iris()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target
df['target_name'] = df['target'].map({0: 'Setosa', 1: 'Versicolor', 2: 'Virginica'})

print("Dataset loaded successfully!")
print("Shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())
print("\nTarget distribution:")
print(df['target_name'].value_counts())
print("\nMissing values:")
print(df.isnull().sum())

```

Dataset loaded successfully!
Shape: (150, 6)

First 5 rows:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

	target	target_name
0	0	Setosa
1	0	Setosa
2	0	Setosa
3	0	Setosa
4	0	Setosa

Target distribution:

```

target_name
Setosa      50
Versicolor  50
Virginica   50
Name: count, dtype: int64

```

Missing values:

```

sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target               0
target_name          0
dtype: int64

```

```
X = df[['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Training samples:", len(X_train))
print("Test samples:", len(X_test))
```

Training samples: 120
Test samples: 30

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Features scaled using StandardScaler")
```

Features scaled using StandardScaler

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
y_pred = knn.predict(X_test_scaled)

accuracy = accuracy_score(y_test, y_pred)
print("Initial KNN model with k=5")
print("Accuracy:", round(accuracy, 4))
```

Initial KNN model with k=5
Accuracy: 1.0

```
k_values = range(1, 21)
cv_scores = []

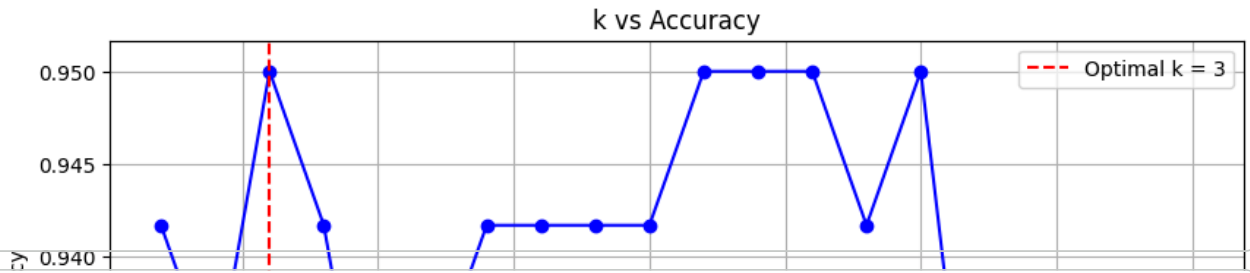
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train_scaled, y_train, cv=5, scoring='accuracy')
    cv_scores.append(scores.mean())

optimal_k = k_values[cv_scores.index(max(cv_scores))]
print("Optimal k:", optimal_k)
print("Best cross-validation accuracy:", round(max(cv_scores), 4))
```

Optimal k: 3
Best cross-validation accuracy: 0.95

```
plt.figure(figsize=(10, 6))
plt.plot(k_values, cv_scores, marker='o', linestyle='-', color='blue')
plt.xlabel('Number of Neighbors (k)')
plt.ylabel('Cross-Validation Accuracy')
plt.title('k vs Accuracy')
plt.axvline(x=optimal_k, color='red', linestyle='--', label=f'Optimal k = {optimal_k}')
plt.legend()
```

```
plt.grid(True)
plt.show()
```



```
knn_final = KNeighborsClassifier(n_neighbors=optimal_k)
knn_final.fit(X_train_scaled, y_train)
y_pred_final = knn_final.predict(X_test_scaled)

print("Final KNN model trained with k =", optimal_k)
```

```
Final KNN model trained with k = 3
```

```
accuracy = accuracy_score(y_test, y_pred_final)
precision = precision_score(y_test, y_pred_final, average='weighted')
recall = recall_score(y_test, y_pred_final, average='weighted')
f1 = f1_score(y_test, y_pred_final, average='weighted')

print("="*50)
print("MODEL PERFORMANCE")
print("="*50)
print("Accuracy:", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall:", round(recall, 4))
print("F1-Score:", round(f1, 4))
print("\nClassification Report:")
print(classification_report(y_test, y_pred_final, target_names=['Setosa', 'Versicolor',
```

```
=====
MODEL PERFORMANCE
=====
```

```
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1-Score: 1.0
```

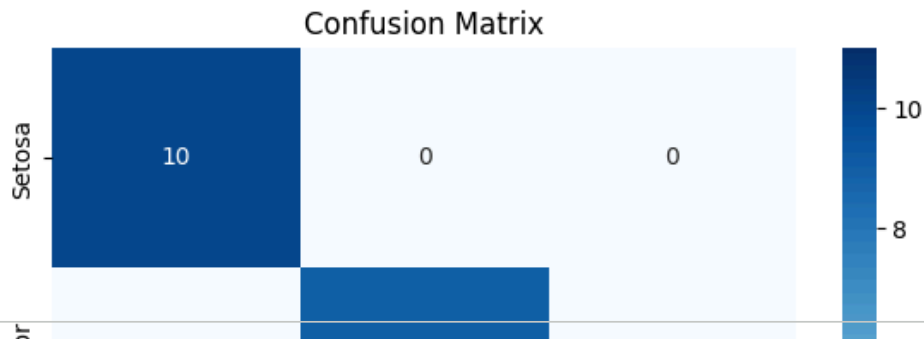
```
Classification Report:
```

	precision	recall	f1-score	support
Setosa	1.00	1.00	1.00	10
Versicolor	1.00	1.00	1.00	9
Virginica	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

```
cm = confusion_matrix(y_test, y_pred_final)
```

```
plt.figure(figsize=(7, 5))
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
             xticklabels=['Setosa', 'Versicolor', 'Virginica'],
             yticklabels=['Setosa', 'Versicolor', 'Virginica'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```



```
new_sample = np.array([[5.1, 3.5, 1.4, 0.2]])
new_sample_scaled = scaler.transform(new_sample)
prediction = knn_final.predict(new_sample_scaled)[0]

print("New sample features:", new_sample[0])
print("Predicted class:", prediction, "(", ['Setosa', 'Versicolor', 'Virginica'][prediction], ")")
```

```
New sample features: [5.1 3.5 1.4 0.2]
Predicted class: 0 ( Setosa )
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X
warnings.warn(
Setosa Versicolor Virginica
Predicted
```

```
print("="*50)
print("PROJECT SUMMARY")
print("="*50)
print("Dataset: Iris (150 samples, 4 features, 3 classes)")
print("Optimal k:", optimal_k)
print("Test Accuracy:", round(accuracy*100, 2), "%")
print("Features scaled: Yes (StandardScaler)")
print("Cross-validation used: Yes (5-fold)")
print("Model ready for new predictions: Yes")
print("Recommendation: Use k =", optimal_k, "for best performance")
```

```
=====
PROJECT SUMMARY
=====
Dataset: Iris (150 samples, 4 features, 3 classes)
Optimal k: 3
Test Accuracy: 100.0 %
Features scaled: Yes (StandardScaler)
Cross-validation used: Yes (5-fold)
Model ready for new predictions: Yes
Recommendation: Use k = 3 for best performance
```

```
print("="*50)
print("BUSINESS INSIGHTS")
print("="*50)
```

```
print("The KNN classifier achieved", round(accuracy*100, 2), "% accuracy on the test se
print("The optimal number of neighbors (k) is", optimal_k, ".")
print("The model can classify iris flowers into 3 species with high accuracy.")
print("Recommendation: Deploy this model for automated iris flower classification.")
```

=====

BUSINESS INSIGHTS

=====

The KNN classifier achieved 100.0 % accuracy on the test set.
The optimal number of neighbors (k) is 3 .
The model can classify iris flowers into 3 species with high accuracy.
Recommendation: Deploy this model for automated iris flower classification.